

📁 Proje: Secure Document Vault (Güvenli Belge Kasası)

- Ders:** İleri Seviye Backend Geliştirme ve Güvenli Kodlama
- Zorluk Seviyesi:** Orta - İleri (Junior+ / Mid Level)
- Rol:** Backend Security Engineer
- Hedef:** Spring Boot ile OWASP standartlarına uygun, saldırıya dayanıklı bir REST API geliştirmek.

1. Senaryo ve Amaç

Bir hukuk bürosu, müşterilerine ait çok hassas dava dosyalarını (sözleşmeler, deliller, kimlikler) dijital ortamda saklamak istiyor. Ancak siber saldırıların arttığı bu dönemde, standart bir dosya yükleme sistemi onlar için yeterli değil.

Göreviniz: Java ve Spring Boot ekosistemini kullanarak, sadece "çalışan" değil, aynı zamanda siber saldırılara karşı "dirençli" bir REST API geliştirmektir. Bu sistemde kimse başkasının dosyasını görememeli, sunucuya virüslü dosya yüklenememeli ve sistem aşırı isteklerle (DDoS/Brute-Force) çökertilememelidir.

2. Teknik Gereksinimler (Tech Stack)

Proje aşağıdaki modern teknolojiler üzerine inşa edilmelidir:

- Dil:** Java 17 veya 21 (Virtual Threads desteği için tercih sebebidir)
- Framework:** Spring Boot 3.2+
- Veritabanı:** PostgreSQL
- Güvenlik:** Spring Security 6 (Method Security aktif)
- IO & Validation:**
 - Apache Tika** (Opsiyonel ama önerilen: Dosya tipi analizi için)
 - Java I/O** (Manuel Magic Bytes kontrolü için)
- Rate Limiting:** Bucket4j
- Utility:** Lombok, Maven/Gradle

3. Fonksiyonel Gereksinimler

Sistemin aşağıdaki 3 temel akışa sahip olması beklenmektedir:

- Auth:** Kullanıcılar sisteme kayıt olabilmeli ve Token (JWT) alarak giriş yapabilmelidir.
- Upload:** Yetkili kullanıcı sisteme PDF formatında belge yükleyebilmelidir.
- View:** Kullanıcı, yüklediği belgenin detaylarını veya içeriğini ID ile sorgulayabilmelidir (GET /api/files/{id}).

4. Güvenli Kodlama Görevleri (Secure Coding Tasks)

Projenin kalbini oluşturan ve seni "sıradan geliştiriciden" ayıran 4 kritik görev şunlardır:

🔒 Görev 1: Yetkilendirme Kontrolü (Broken Access Control)

- Problem:** Standart bir yazılımda, User A giriş yaptığında, URL'deki ID'yi değiştirerek (/api/files/125) User B'nin dosyasını görüntüleyebilir (IDOR Açığı).
- Yasak:** Service katmanında if (file.getOwner().equals(currentUser)) gibi manuel if-else blokları yazmak yasaktır (Spagetti koda yol açar).
- İstenen Çözüm:**

- Spring Security'nin **AOP (Aspect Oriented Programming)** yeteneklerini kullanın.
- `@EnableMethodSecurity` anotasyonunu aktif edin.
- Controller veya Service metodunda `@PostAuthorize` anotasyonunu kullanarak, kuralı deklaratif olarak belirtin: `@PostAuthorize("returnObject.owner == authentication.name")`

🛡️ Görev 2: Gerçek Dosya Tipi Doğrulaması (Malicious File Upload)

- **Problem:** Saldırganlar, zararlı `_yazilim.exe` dosyasının adını `tez_odevi.pdf` olarak değiştirip sisteme yükleyebilirler. Sadece dosya uzantısına (`.pdf`) bakmak yetersizdir.
- **İstenen Çözüm:**
 - **Seviye 1 (Öğrenme):** Java `InputStream` kullanarak dosyanın ilk 4 byte'ını (Magic Bytes) okuyun ve PDF imzası (`0x25 0x50 0x44 0x46`) ile eşleşip eşleşmediğini kontrol edin.
 - **Seviye 2 (Profesyonel):** Projeye **Apache Tika** kütüphanesini ekleyin ve dosya içeriğinin MIME type'ını (`application/pdf`) kesin olarak doğrulayın.

🛡️ Görev 3: Hız Sınırlama (Rate Limiting)

- **Problem:** Bir bot, saniyede 1000 istek göndererek sistemi kilitleyebilir (DoS Saldırısı).
- **İstenen Çözüm:**
 - **Bucket4j** kütüphanesini kullanarak bir Filter (Interceptor değil, Filter tercih edilir) yazın.
 - Her IP adresi için dakikada maksimum **10 istek** hakkı tanımlayın.
 - Limit aşıldığında sistem veritabanına gitmeden HTTP 429 (Too Many Requests) hatası dönmelidir.

🛡️ Görev 4: Güvenli Hata Yönetimi (Security Misconfiguration)

- **Problem:** Sistem hata verdiğinde (örneğin IDOR yakalandığında), Spring Boot varsayılan olarak "Stack Trace" (kodun hangi satırda hata verdiğini) bilgisini döner. Bu, saldırganlara sistem hakkında ipucu verir.
- **İstenen Çözüm:**
 - Global bir **Exception Handler** (`@ControllerAdvice`) yazın.
 - `AccessDeniedException` veya `SecurityException` fırlatıldığında, kullanıcıya sadece şu JSON dönmelidir:

JSON

```
{
  "error": "Erişim Reddedildi",
  "message": "Bu işlem için yetkiniz yok.",
  "timestamp": "2026-01-15T14:30:00"
}
```

5. Proje Yol Haritası (Adım Adım Uygulama)

1. **Gün 1: Temel İskelet**
 - Spring Initializr ile projeyi kur.
 - PostgreSQL bağlantısını yap.
 - User ve Document entity'lerini oluştur.
2. **Gün 2: İş Mantığı**
 - Dosya yükleme (Upload) ve indirme servislerini güvenli haliyle yaz. Çalıştığını gör.
3. **Gün 3: Hata Yönetimi (Görev 4)**
 - `@ControllerAdvice` sınıfını oluştur. Tüm hataları yakaladığından emin ol.
4. **Gün 4: Dosya Güvenliği (Görev 2)**
 - Upload servisine Magic Bytes (veya Apache Tika) kontrolünü ekle. `.exe` dosyasını `.pdf` yapıp yüklemeyi dene, sistemin reddettiğini gör.
5. **Gün 5: Erişim Kontrolü (Görev 1)**
 - `@PostAuthorize` ekle. İki farklı kullanıcı oluştur. Biriyle token alıp diğersinin dosya ID'sini çağır. 403 Forbidden (veya senin özel JSON hatanı) aldığını gör.
6. **Gün 6: Rate Limiting (Görev 3)**
 - Bucket4j filtresini ekle. Postman ile üst üste hızlı istekler atarak 429 hatasını gör.

6. Proje Dosya Yapısı Örneği

Spring Boot'ta dosyaları nereye koyacağın konusunda kafan karışırsa bu yapıyı baz alabilirsin:

Plaintext

src/main/java/com/seninadin/securevault

